Module 14 - chapter 9

Backup EC2 Volumes: Automate creating Snapshots

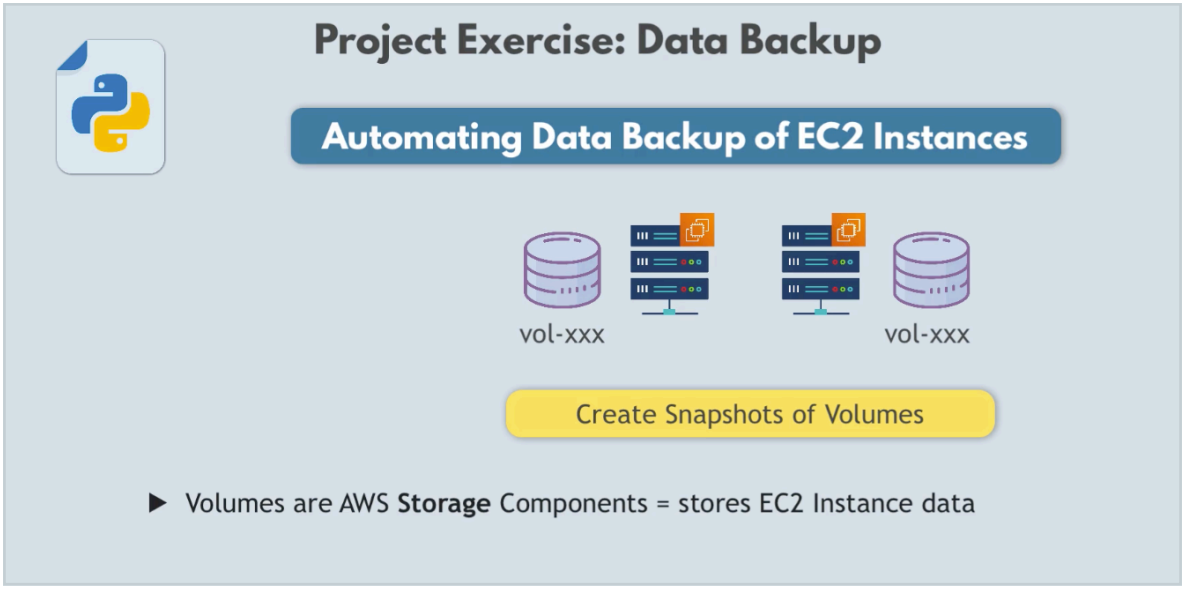
The project file used in this lecture can be found here:

- https://gitlab.com/twn-devops-bootcamp/latest/14-automation-with-python/automation-projects/-/blob/main/volume-backups.py

Data Backup

Project intro

Automate Data Backup for EC2 instances -> we will snapshots of Volumes for EC2 instances

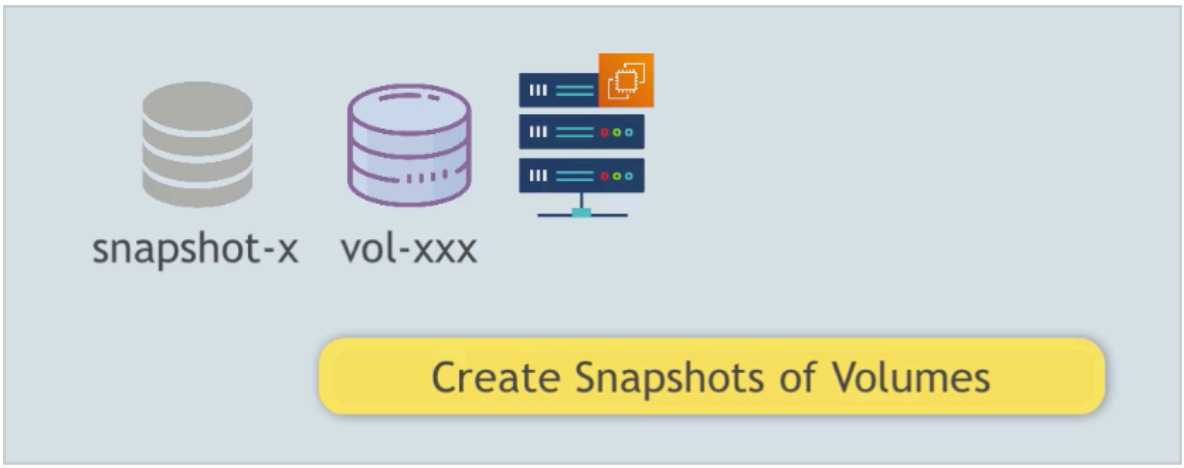


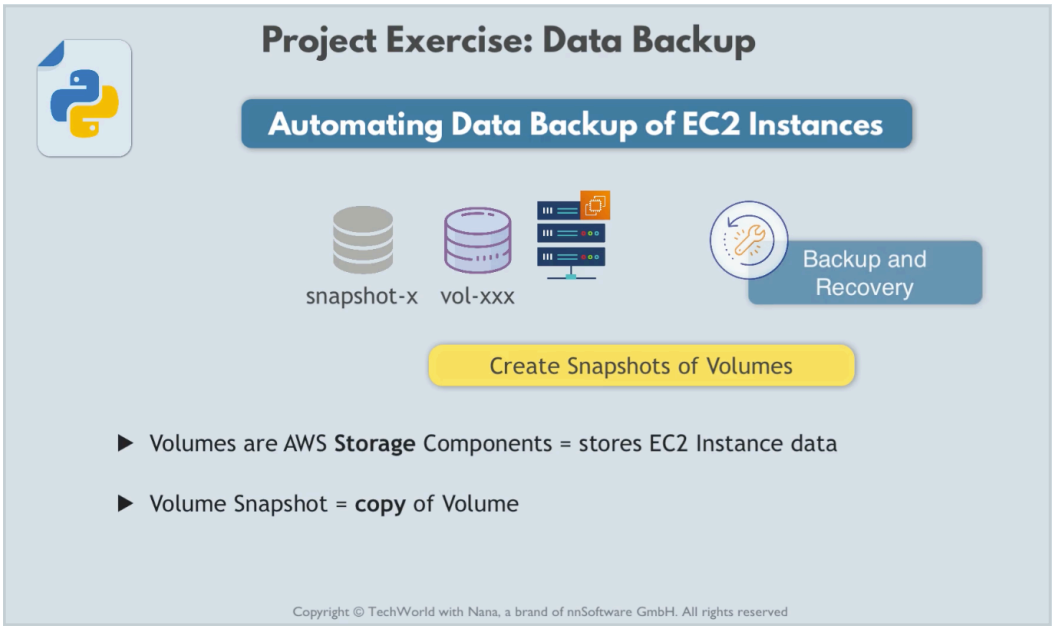
Volume -> A volume of an EC2 instance is like a hard drive for the EC2 server.

So when we create an EC2 instance a volume gets created automatically and hen we delete an EC2 instance the volume gets deleted as well.

Snapshot -> a copy

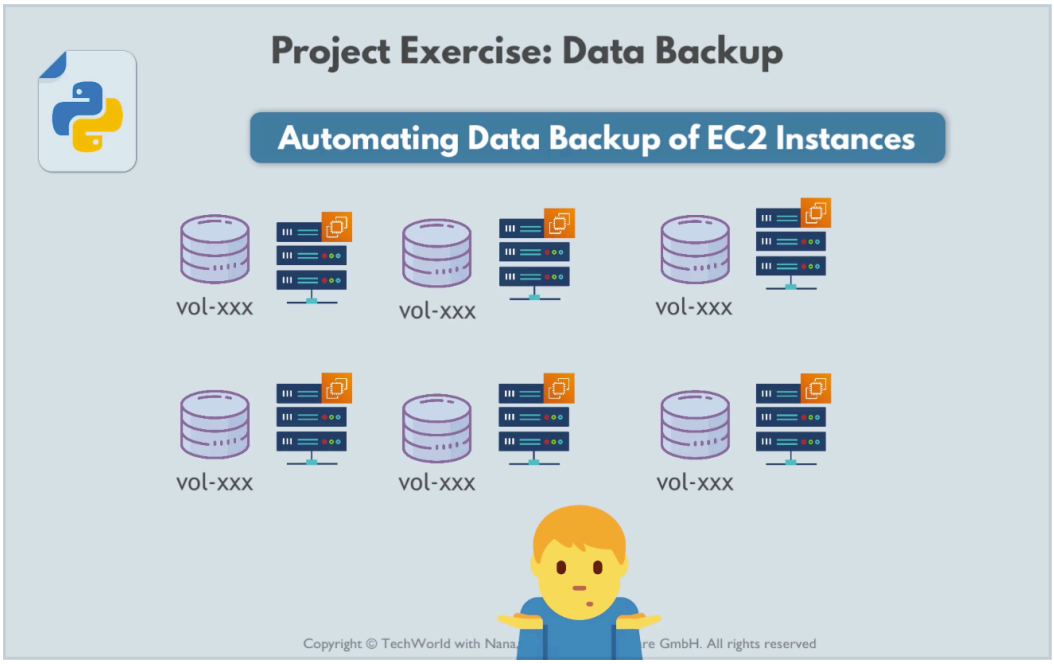
Snapshot of volume -> copy of a volume at the moment of taking the snapshot.



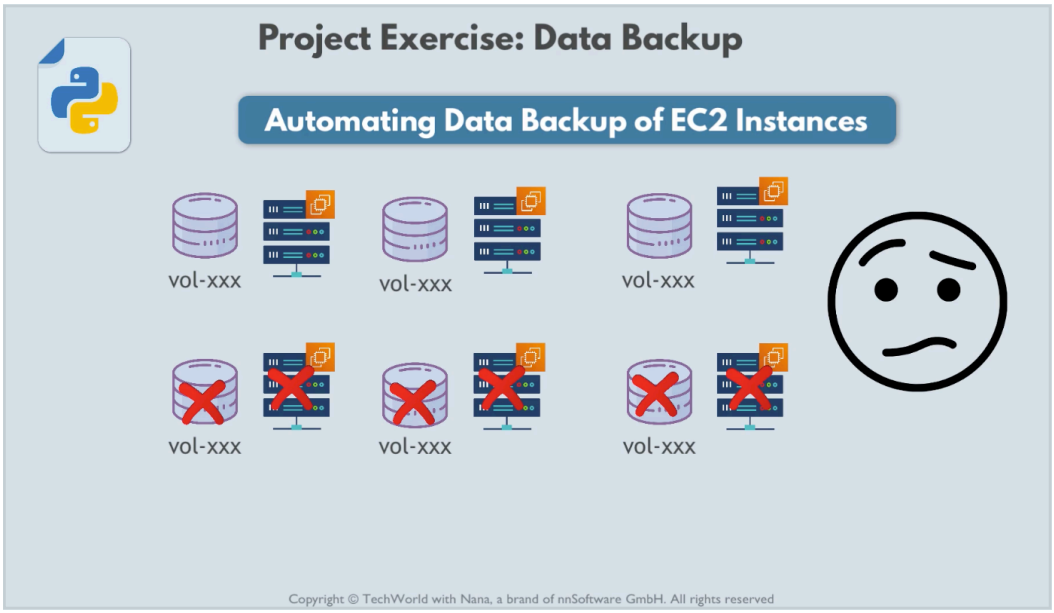


Making snapshots and storing them needs to be automated to make sure that we have backups.

Now if we have 50 EC2 volumes that have not been backedUp



And then some EC2 instances crashed and we lost the data



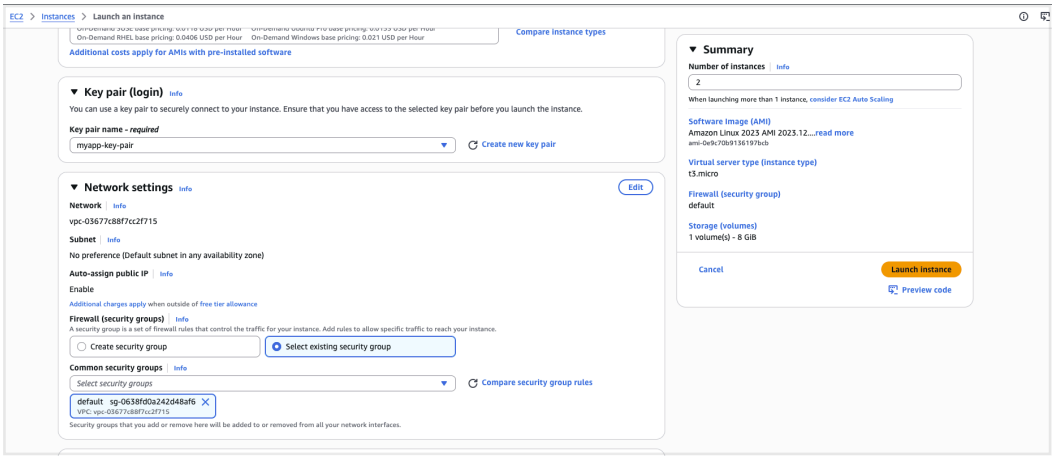
So now the team thinks is important to make daily backups to avoid this in the future.

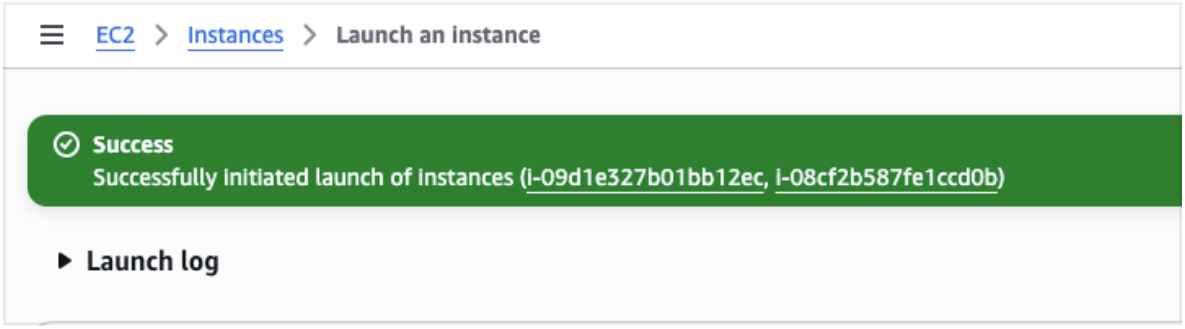
And now is our job to do the backups and we will automate the creation of the backups so we don't have to do it ourselves every single day.

We will make a python program to run the backups every day at a specific time.

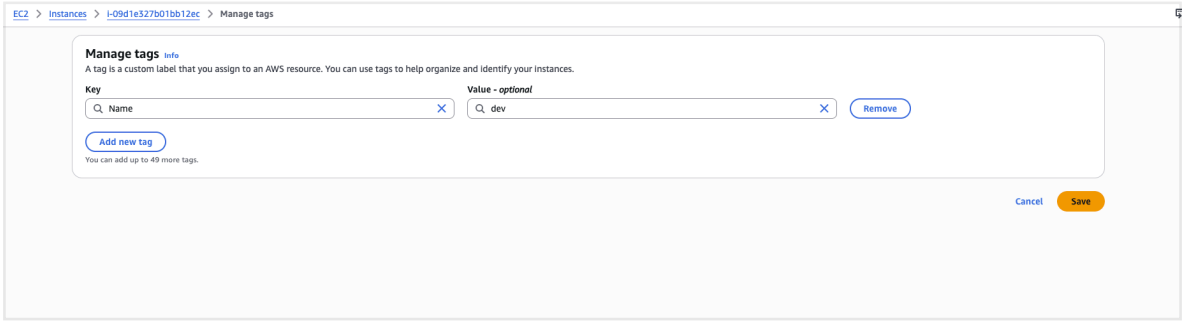
Create 2 EC2 instances

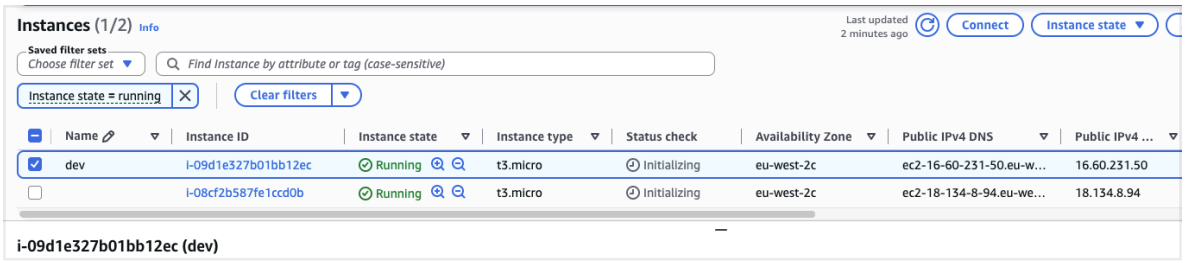
We will launch 2 servers manually one with tag dev and the other with tag prod



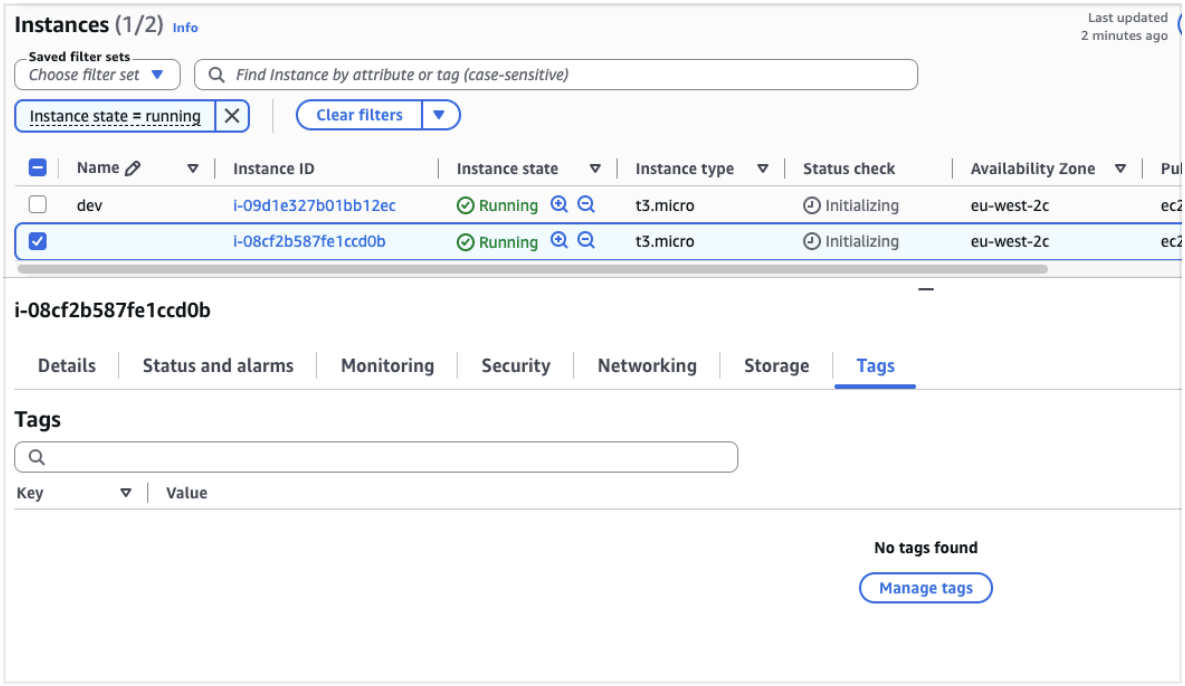


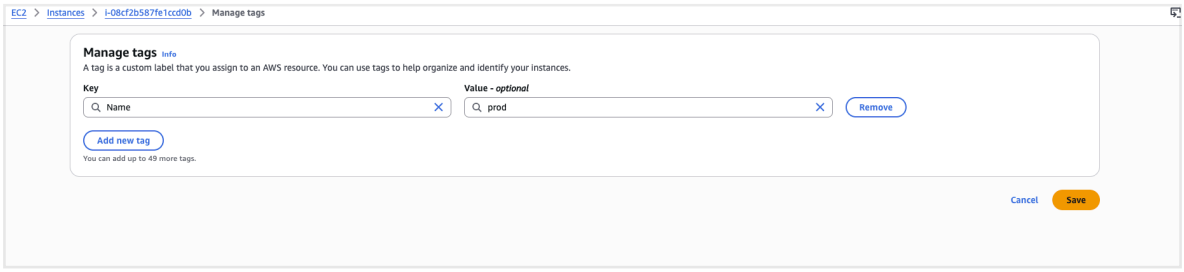
And we add the ‘dev’ tag

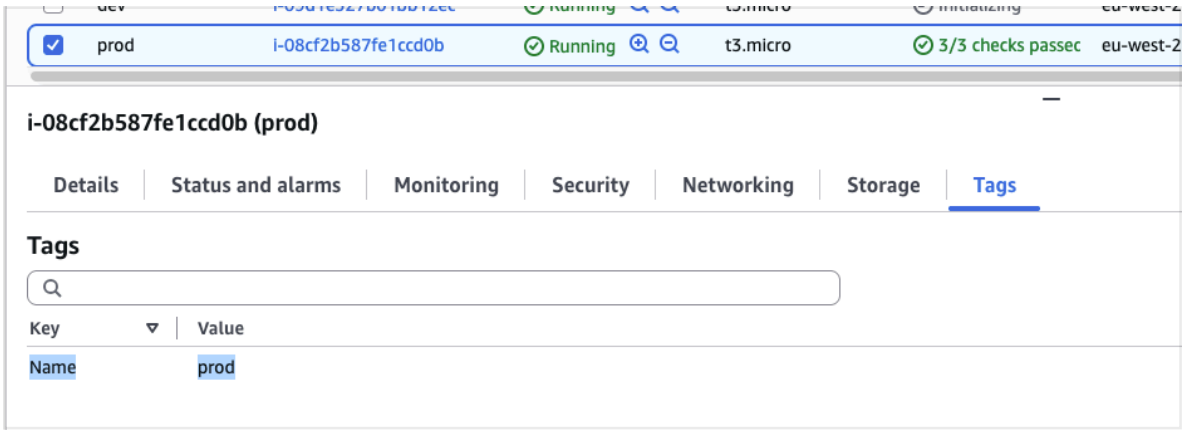




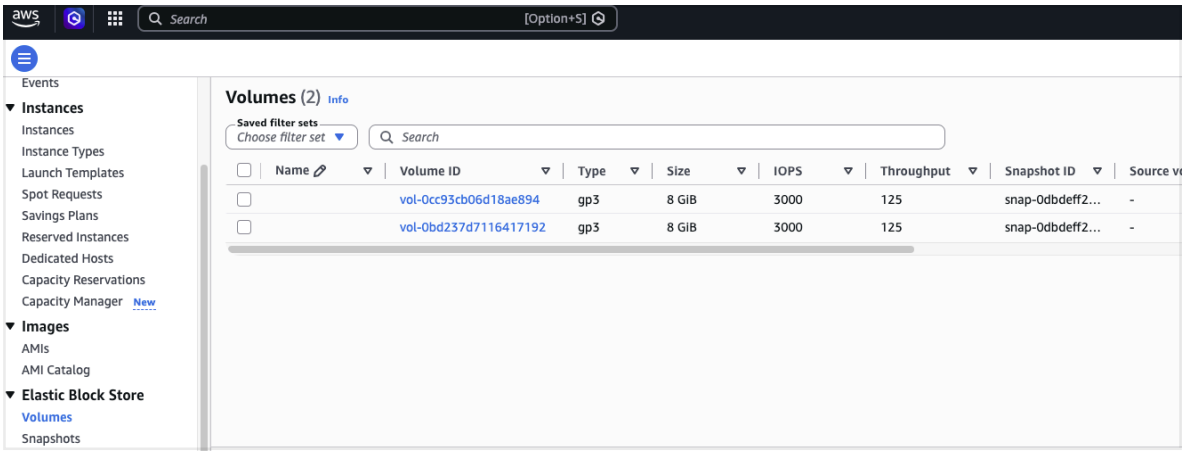
And the ‘prod’ tag



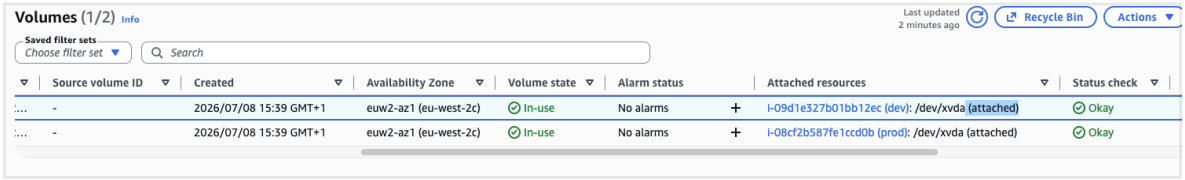




Now let's go to volumes

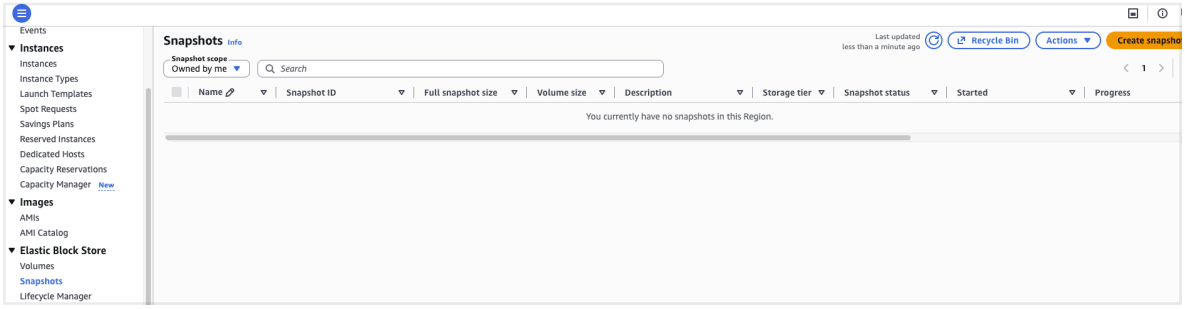


We can see two volumes, one for each EC2 server and here we can see the values are attached to the EC2 instances



And each volume have a size of 8GiB and they are of type gp3 -> think of them as hard drives attached to each server.

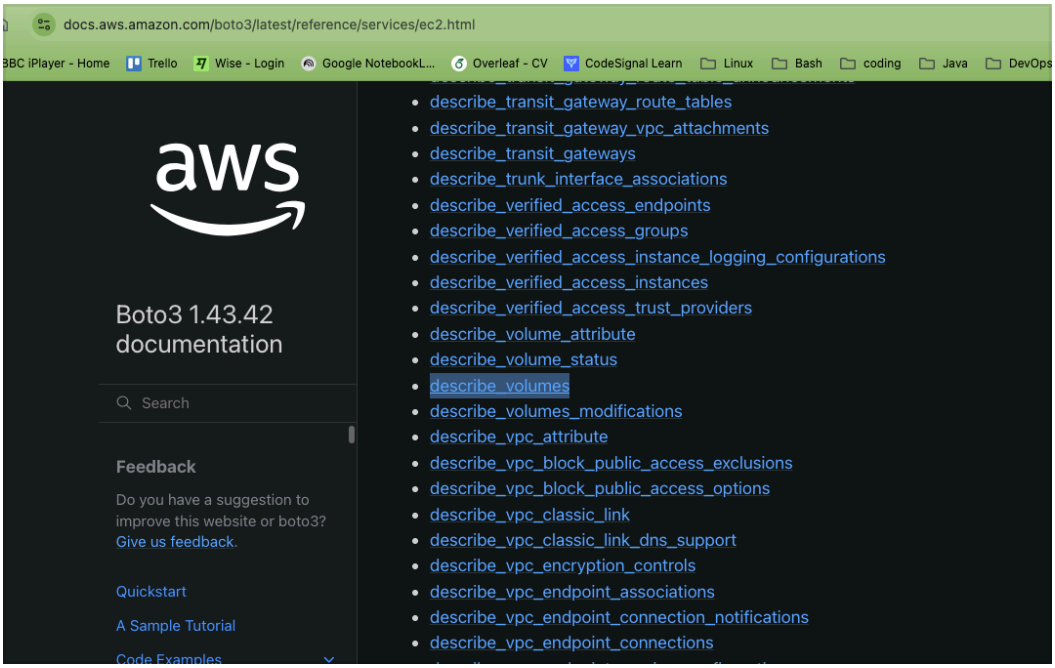
Now let's look at snapshot and it is empty



We will write a schedule task that every day collects the volume and creates a snapshot/backup to each volume.

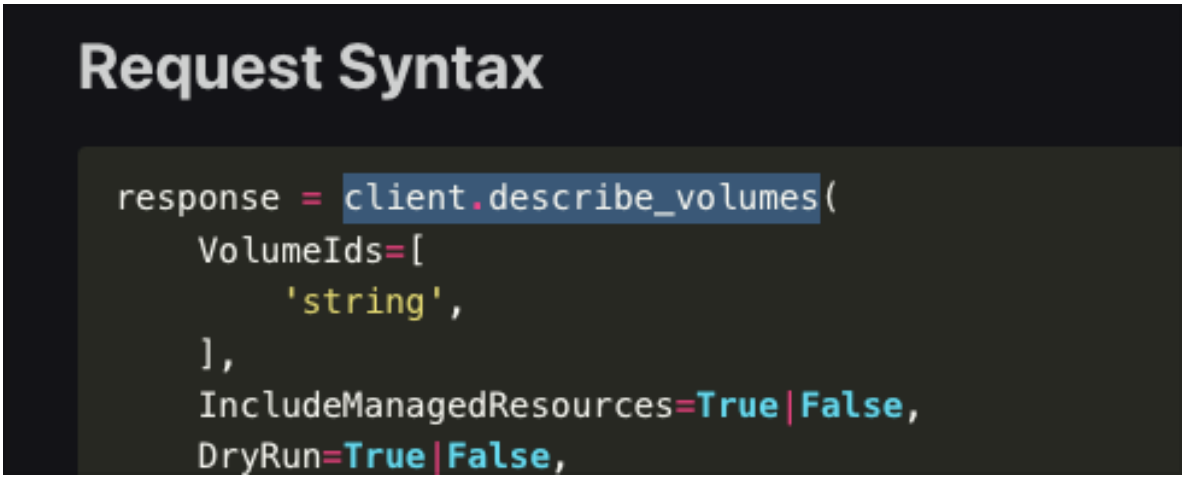
Backup implementation

In AWS documentation we go back to EC2 service and we can find a function called describe_volumes()
https://docs.aws.amazon.com/boto3/latest/reference/services/ec2/client/describe_volumes.html

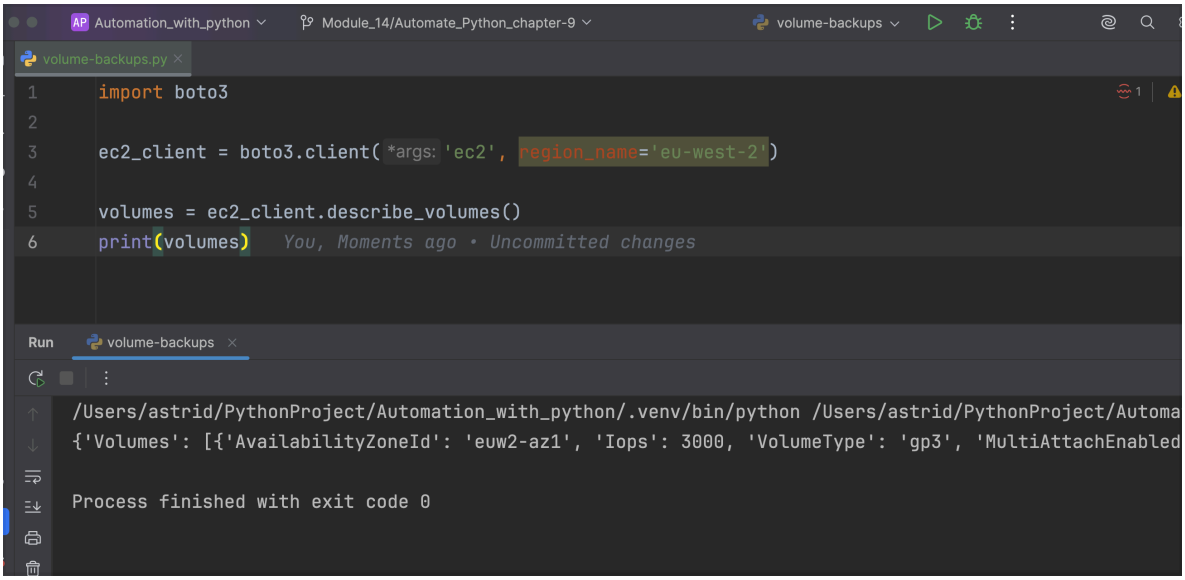


And returns a list of volumes with information like the VolumeId which we will need to create the snapshot later

The describe_volumes is created on an EC2 client

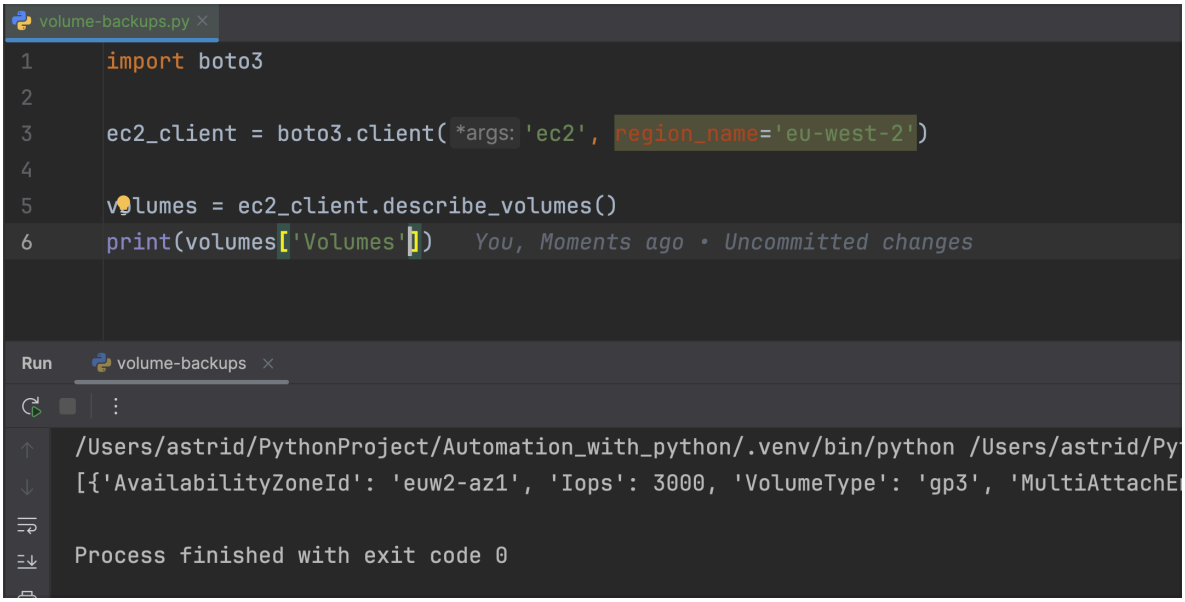


So here is the beginning of the program:



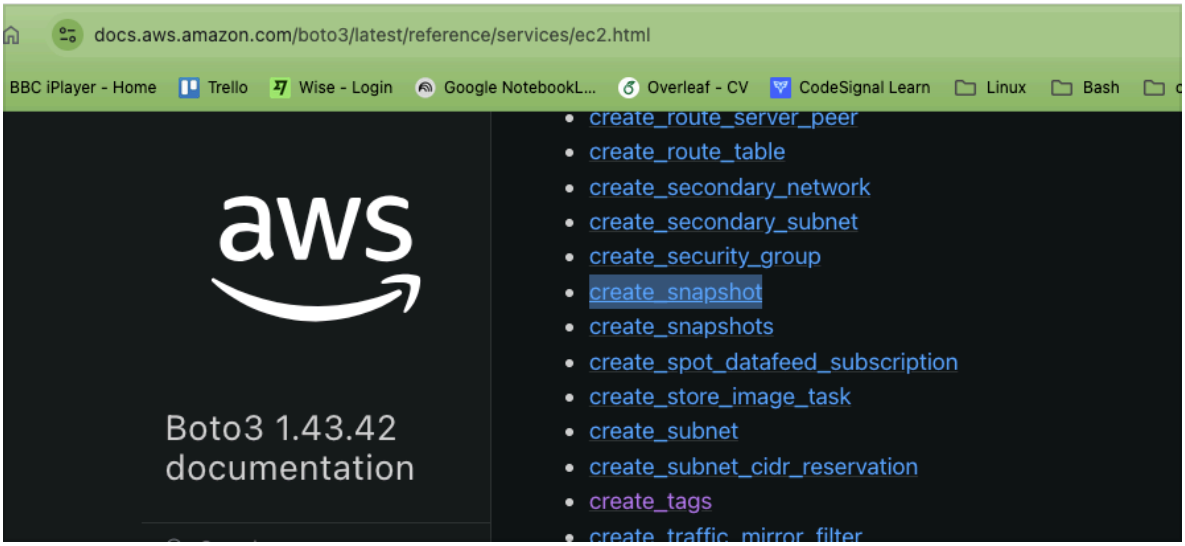
But we only want the volumes

-> volumes['Volumes’]

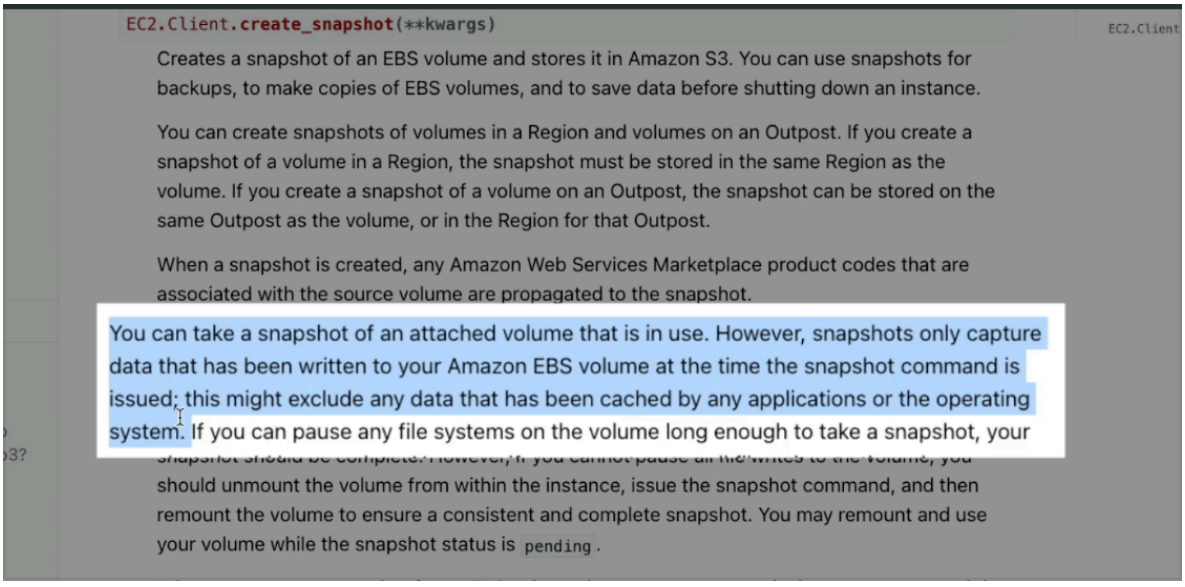


Now we iterate each volume (we have 2 in this demo)

And now we can create a snapshot during the iteration with function create_snapshot()
https://docs.aws.amazon.com/boto3/latest/reference/services/ec2/client/create_snapshot.html

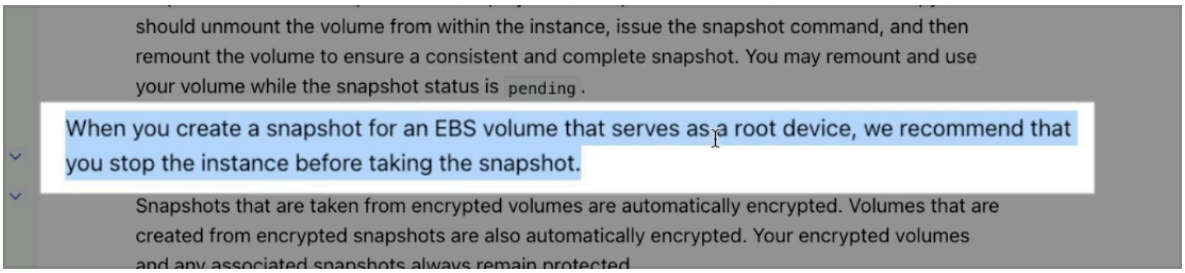


Important notes to do the snapshot: ‼️



If the volume is still in use the snapshot might not include temporarily stored data like application data.

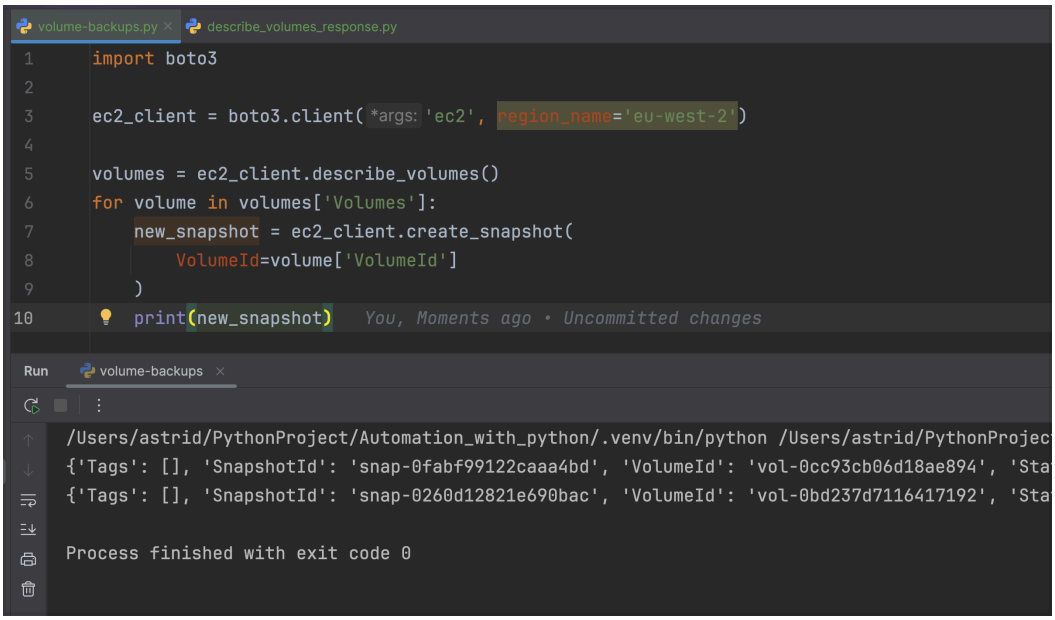
And



If the volume is the root device of an instance then the Instance should be stop before creating the snapshot to make sure data is consistent.

In this DEMO we won't deal with this at is not part of the scope of the DEMO but it is important for prod environments. ‼️

So we update our code:



We got 2 snapshots one per server.

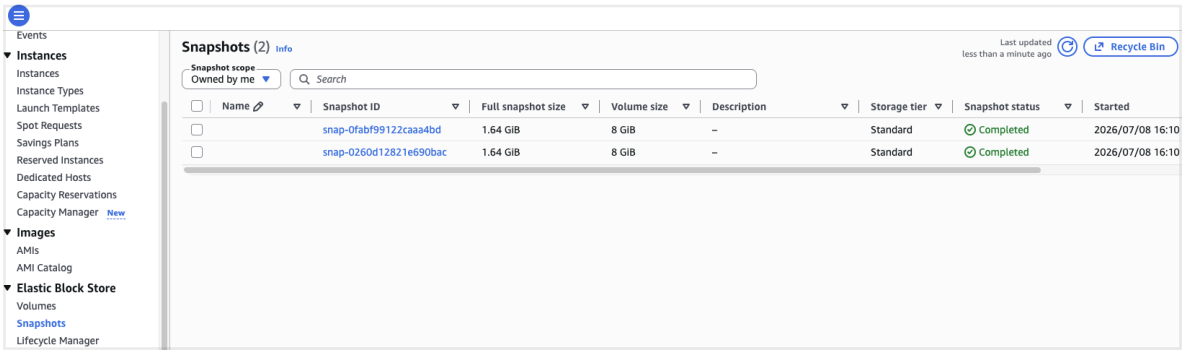
And this is the structure of what we got back per server:

RETURNS:

Response Syntax

```
{
    'OwnerAlias': 'string',
    'OutpostArn': 'string',
    'Tags': [
        {
            'Key': 'string',
            'Value': 'string'
        },
    ],
    'StorageTier': 'archive'|'standard',
    'RestoreExpiryTime': datetime(2015, 1, 1),
    'SseType': 'sse-ebs'|'sse-kms'|'none',
    'AvailabilityZone': 'string',
    'TransferType': 'time-based'|'standard',
    'CompletionDurationMinutes': 123,
    'CompletionTime': datetime(2015, 1, 1),
    'FullSnapshotSizeInBytes': 123,
    'SnapshotId': 'string',
    'VolumeId': 'string',
    'State': 'pending'|'completed'|'error'|'recoverable'|'recovering',
    'StateMessage': 'string',
    'StartTime': datetime(2015, 1, 1),
    'Progress': 'string',
    'OwnerId': 'string',
    'Description': 'string',
    'VolumeSize': 123,
    'Encrypted': True|False,
    'KmsKeyId': 'string',
    'DataEncryptionKeyId': 'string'
}
```

And we check AWS console and we can see the 2 new snapshots



Scheduler implementation

Import the schedule library and set the schedule object to execute the function that contains the creation of the snapshot.

```python
import boto3
import schedule

ec2_client = boto3.client( *args: 'ec2', region_name='eu-west-2')

def create_volume_snapshots():  1 usage  new *
    volumes = ec2_client.describe_volumes()
    for volume in volumes['Volumes']:
        new_snapshot = ec2_client.create_snapshot(
            VolumeId=volume['VolumeId']
        )
        print(new_snapshot)

schedule.every().day.do(create_volume_snapshots)    You, Moments ago • Uncomm

# start the scheduler
while True:
    schedule.run_pending()
```

To test the program let’s change days to seconds:

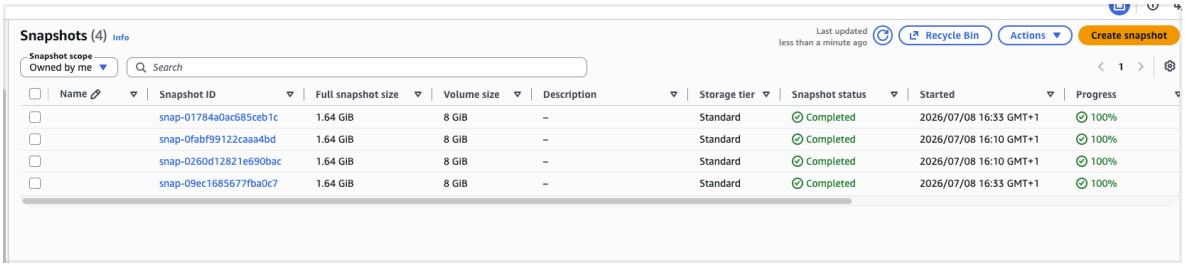
```python
        print(new_snapshot)

schedule.every(5).seconds.do(create_volume_snapshots)

# start the scheduler
while True:
    schedule.run_pending()   You, A minute ago • Uncommitted changes
```

```
Run  volume-backups
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Automation_
{'Tags': [], 'SnapshotId': 'snap-01784a0ac685ceb1c', 'VolumeId': 'vol-0cc93cb06d18ae894', 'State': 'pendin
{'Tags': [], 'SnapshotId': 'snap-09ec1685677fba0c7', 'VolumeId': 'vol-0bd237d7116417192', 'State': 'pendin
```

Now we have 4 snapshots, 2 per server



And an error after that
“botocore.exceptions.ClientError: An error occurred (SnapshotCreationPerVolumeRateExceeded) when calling the CreateSnapshot operation: **The maximum per volume CreateSnapshot request rate has been exceeded**. Use an increasing or variable sleep interval between requests.”

```
core/client.py", line 1094, in _make_api_call

lling the CreateSnapshot operation: The maximum per volume CreateSnapshot request rate has been exceeded. l
```

We tried to create a new snapshot when the previous one was still in creation. So we need more time between triggering the creation of snapshots.

But for our DEMO this is good enough. In reality it will be once a day so there will be no issue regarding timing.

Now let's say we only want to backup production server volumes -> we need to filter the volumes.

So when we get the volumes with describe_volumes() we can filter them as per documentation:

Request Syntax

```
response = client.describe_volumes(
    VolumeIds=[
        'string',
    ],
    IncludeManagedResources=True|False,
    DryRun=True|False,
    Filters=[
        {
            'Name': 'string',
            'Values': [
                'string',
            ]
        },
    ],
    NextToken='string',
    MaxResults=123
)
```

And these are the values that the filter can use, one of which is 'tag:<key>'

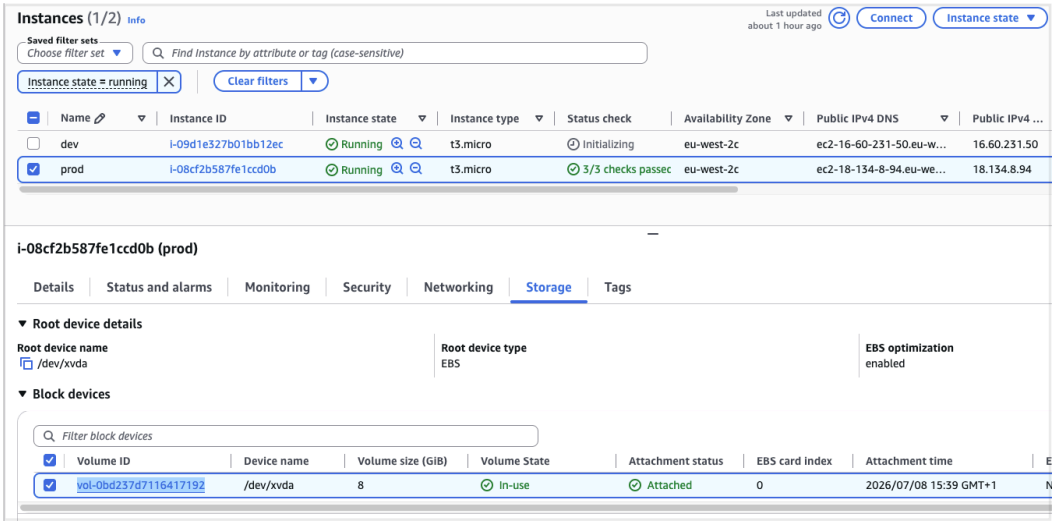
- **Filters** (*list*) –

  The filters.

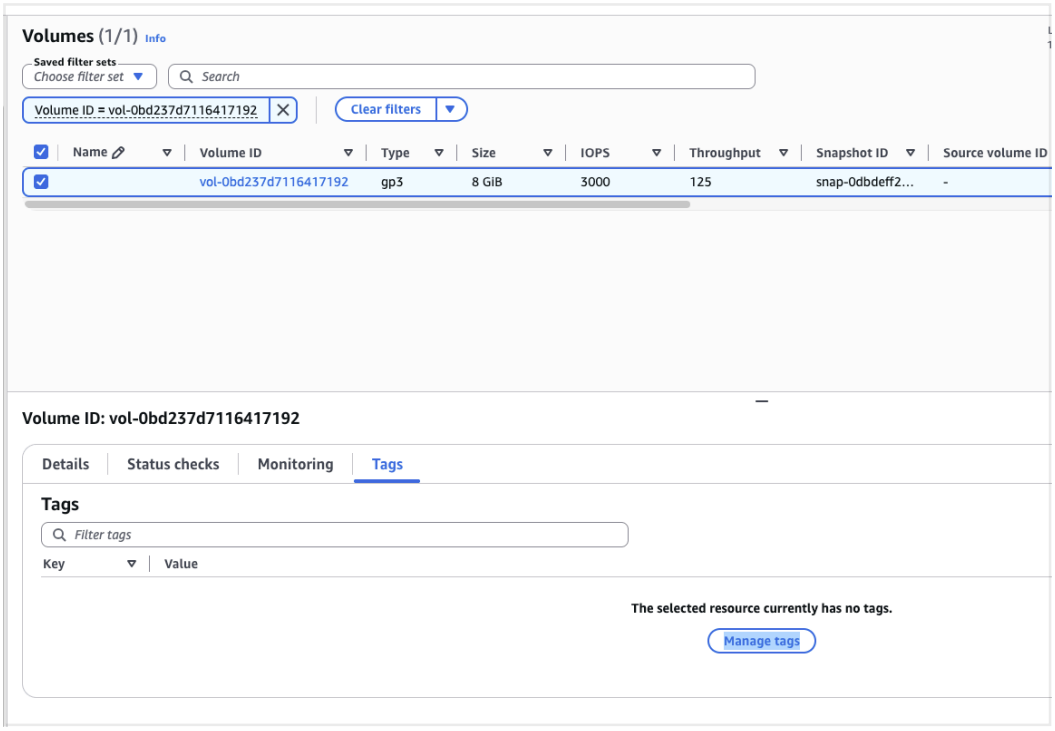
  - `attachment.attach-time` - The time stamp when the attachment initiated.
  - `attachment.delete-on-termination` - Whether the volume is deleted on instance termination.
  - `attachment.device` - The device name specified in the block device mapping (for example, `/dev/sda1`).
  - `attachment.instance-id` - The ID of the instance the volume is attached to.
  - `attachment.status` - The attachment state (`attaching` | `attached` | `detaching`).
  - `availability-zone` - The Availability Zone in which the volume was created.
  - `availability-zone-id` - The ID of the Availability Zone in which the volume was created.
  - `create-time` - The time stamp when the volume was created.
  - `encrypted` - Indicates whether the volume is encrypted (`true` | `false`)
  - `fast-restored` - Indicates whether the volume was created from a snapshot that is enabled for fast snapshot restore (`true` | `false`).
  - `multi-attach-enabled` - Indicates whether the volume is enabled for Multi-Attach (`true` | `false`)
  - `operator.managed` - A Boolean that indicates whether this is a managed volume.
  - `operator.principal` - The principal that manages the volume. Only valid for managed volumes, where `managed` is `true`.
  - `size` - The size of the volume, in GiB.
  - `snapshot-id` - The snapshot from which the volume was created.
  - `status` - The state of the volume (`creating` | `available` | `in-use` | `deleting` | `deleted` | `error`).
  - `tag:<key>` - The key/value combination of a tag assigned to the resource. Use the tag key in the filter name and the tag value as the filter value. For example, to find all resources that have a tag with the key `Owner` and the value `TeamA`, specify `tag:Owner` for the filter name

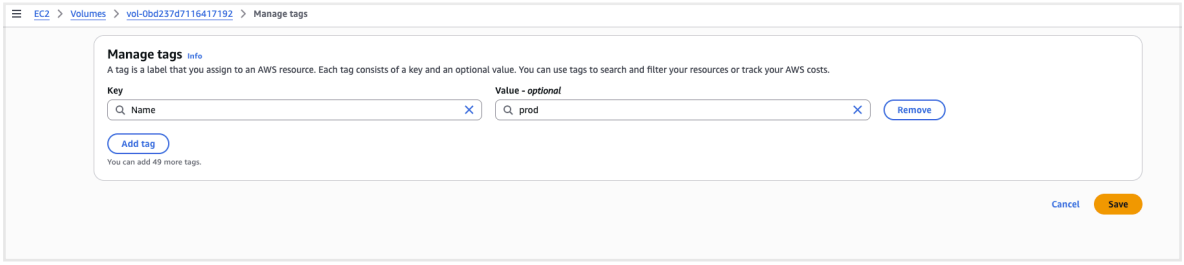
So we need our volumes to have tags to be able to use this filter. The best way is automatically assign tags to the volumes with a python script but for this Demo we will add the tag to the prod volume manually:

So I select the prod instance and then in the storage tab I select the volume

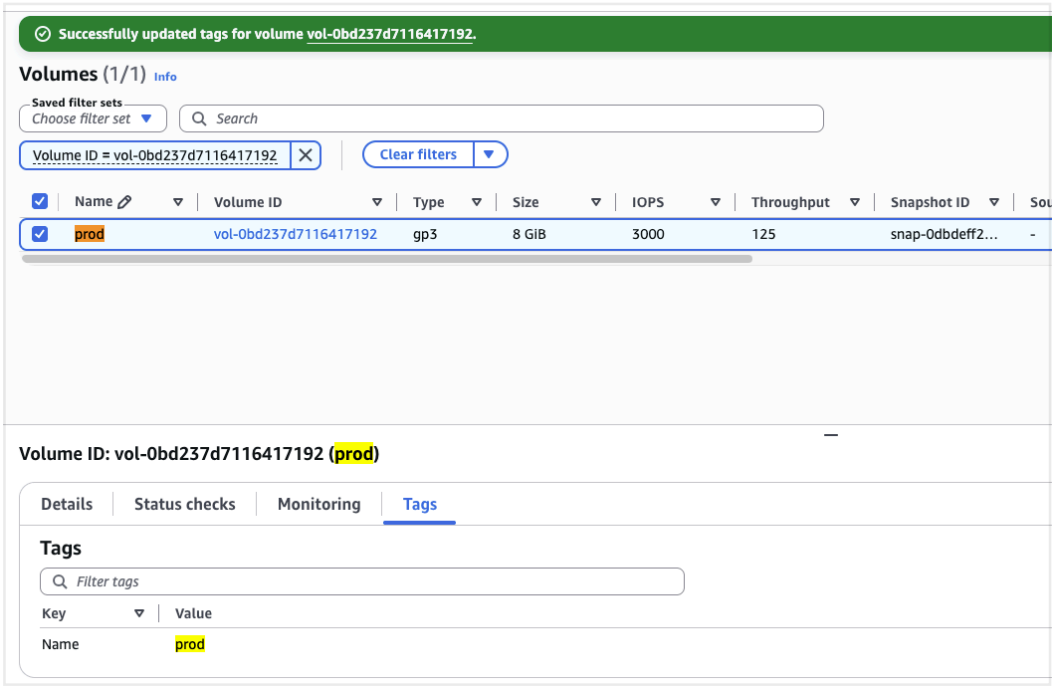


Once I am inside the volume section I go to the tags tab and add the tag:

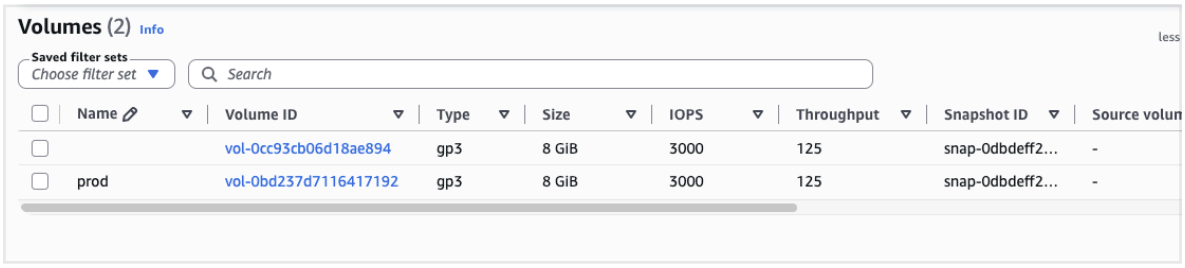




And it has the tag now:



And we have one volume with a tag and another without tag



Next we can filter in our python program

```
6   def create_volume_snapshots():  1 usage   new *
7       volumes = ec2_client.describe_volumes(
8           Filters=[
9               {
10                  'Name': '',
11                  'Values': ['']
12              }
13          ]
14      )
15      for volume in volumes['Volumes']:    You, A minute ag
```

using tag with key ‘Name’ and value ‘prod’

```
    volumes = ec2_client.describe_volumes(
        Filters=[
            {
                'Name': 'tag:Name',
                'Values': ['prod']   You, Moments ago •
            }
        ]
    )
```

And if we were to get prod and staging environments we can do this:

```
def create_volume_snapshots():
    volumes = ec2_client.describe_volumes(
        Filters=[
            {
                'Name': 'tag:Name',
                'Values': ['prod', 'staging']
            }
        ]
    )
```

And the tag key could be ‘Environment’ instead of ’Name’

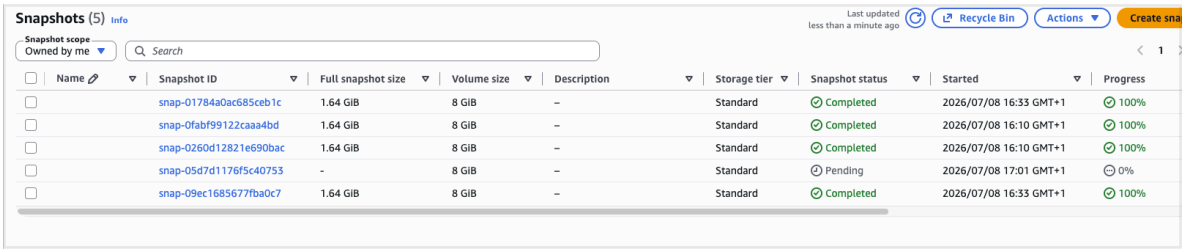
Let’s try again but using 20 seconds to avoid the timing error:

```
        )
        print(new_snapshot)

schedule.every(20).seconds.do(create_volume_snapshots)   You, 3 minutes ago • Uncommitted changes

# start the scheduler
while True:
    schedule.run_pending()
```

```
Run   volume-backups
/Users/astrid/PythonProject/Automation_with_python/.venv/bin/python /Users/astrid/PythonProject/Automat
{'Tags': [], 'SnapshotId': 'snap-05d7d1176f5c40753', 'VolumeId': 'vol-0bd237d7116417192', 'State': 'per
```



So now we are creating snapshots only for prod volumes.